DAY 6

Categorical Column Analysis

Project: Customer Churn Prediction & LTV Engine

Role: Member 2 | Date: 18-05-2026

Objective

Analyse all text-based (categorical) columns in the Telco Customer Churn dataset. Identify which columns are Binary (2 options) and which are Multi-Category (3+ options), then decide the correct encoding method for each — without applying any encoding yet.

What is Categorical Encoding?

Machine Learning models can only work with numbers — they cannot understand words like **'Yes'**, **'No'**, or **'Month-to-month'**. Categorical Encoding is the process of converting these text values into numbers. Before we can encode, we first need to understand what types of text columns we have.

Encoding Type	When to Use	Example
Label Encoding	Binary columns (only 2 options)	Yes=1, No=0
One-Hot Encoding	Multi-category (3+ options)	DSL, Fiber, No → 3 columns

File Created

```
notebooks/feature_engineering/DAY_6_(18-05-26)/01_categorical_encoding.ipynb
```

Step-by-Step Implementation

Step 1

Open Jupyter Notebook

In VS Code left panel navigate to:

notebooks/feature_engineering/DAY_6_(18-05-26)/

Open the file: 01_categorical_encoding.ipynb

Select Python kernel (top-right corner of notebook).

```
import pandas as pd
import numpy as np
```

Step 2

Import Libraries — Cell 1

Click + Code and type:

```
df = pd.read_csv('data/raw/telco_raw.csv')
print(f'Shape: {df.shape}')
df.head()
```

Step 3

Load the Dataset — Cell 2

Load the raw CSV file from the data folder:

```
print(df.columns.tolist())
```

Step 4

View All Column Names — Cell 3

Print all 21 column names to understand the dataset structure.

```
cat_cols = df.select_dtypes(include='object').columns
print(f'Total categorical columns: {len(cat_cols)}')
for col in cat_cols: print(f' - {col}')
```

Step 5

Find Categorical Columns — Cell 4

Use select_dtypes(include='object') to get text columns.

Print total count and list of all categorical columns.

```
for col in cat_cols:
print(f'{col}: {df[col].nunique()} unique')
print(f' Values: {df[col].unique()}')
```

Step 6

Count Unique Values — Cell 5

Loop through each categorical column.

Print how many unique values each column has.

Example: gender has 2 unique values: Male, Female.

```
binary_cols, multi_cols = [], []
for col in cat_cols:
if df[col].nunique() == 2: binary_cols.append(col)
else: multi_cols.append(col)
print('Binary:', binary_cols)
print('Multi :', multi_cols)
```

Step 7

Separate Binary vs Multi-Category — Cell 6

Binary = exactly 2 unique values → use Label Encoding.

Multi-Category = 3 or more values → use One-Hot Encoding.

Store results in binary_cols and multi_cols lists.

```
strategy_data = []
for col in binary_cols:
strategy_data.append({'Column':col,'Type':'Binary','Method':'Label Encoding'})
for col in multi_cols:
strategy_data.append({'Column':col,'Type':'Multi','Method':'One-Hot Encoding'})
strategy_df = pd.DataFrame(strategy_data)
print(strategy_df.to_string(index=False))
```

Step 8

Build Encoding Strategy Table — Cell 7

Create a pandas DataFrame showing each column and its encoding method.

This is your planning document — no encoding is done yet.

```
missing = df[cat_cols].isnull().sum()
print(missing)
```

Step 9

Check Missing Values — Cell 8

Check isnull().sum() for all categorical columns.

Identify columns that may need cleaning before encoding.

```
strategy_df.to_csv(
'docs/reports/DAY_6_(18-05-26)/encoding_strategy.csv',
index=False)
print('Saved!')
```

Step 10

Save Strategy CSV — Cell 9

Save the encoding strategy table as a CSV reference file.

Path: docs/reports/DAY_6_(18-05-26)/encoding_strategy.csv

TIP: Always run cells from top to bottom (Cell 1 first, then 2, 3 ...). If you skip Cell 1, Cell 2 will fail with NameError: name 'pd' is not defined.

Day 6 Deliverables

File	Location	Purpose
01_categorical_encoding.ipynb	notebooks/feature_engineering/DAY_6_(18-05-26)/	Main analysis notebook
encoding_strategy.csv	docs/reports/DAY_6_(18-05-26)/	Encoding plan reference